

15

Lecture 15

**Classic Synchronization Problems II: Dining
Philosophers and Monitors**

by Brijesh Shukla
C07: Operating Systems

2

Join the lecture online on your dashboard

Today's Agenda

- Review of Producer-Consumer
- Dining Philosophers Problem
- Semaphore Solutions for Philosophers
- Introduction to Monitors
- Condition Variables in Monitors
- Monitor Examples and Comparison



Review of Producer-Consumer Problem

✚ What is it?

- A classic **synchronization problem**
- Demonstrates **process coordination** using a **shared buffer**

🏠 Scenario

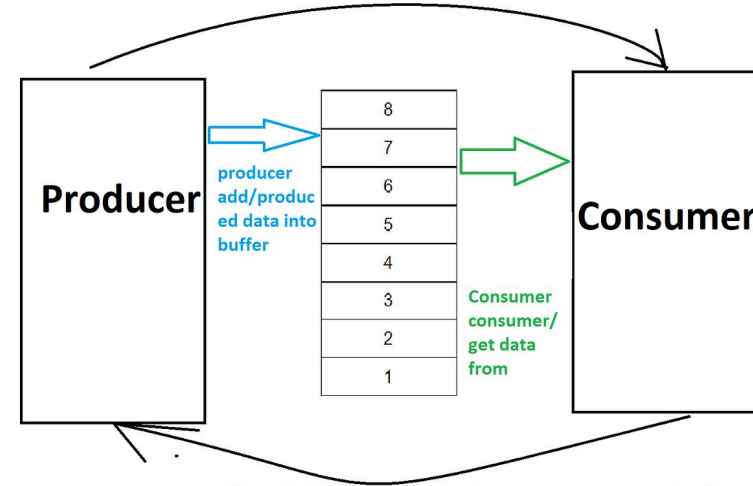
-  **Producer**: creates data → puts into buffer
-  **Consumer**: removes data → processes it
-  **Buffer**: limited size (**bounded buffer**)

⚠️ Key Challenges

-  Producer waits if **buffer is full**
-  Consumer waits if **buffer is empty**
-  **Mutual exclusion** required for buffer access
-  Prevent **race conditions**

🔑 Synchronization Requirements

-  Only **one process** accesses buffer at a time
-  Producer **sleeps** when buffer is full
-  Consumer **sleeps** when buffer is empty
-  Waiting process **wakes up** when condition changes



Producer-Consumer Semaphore Solution

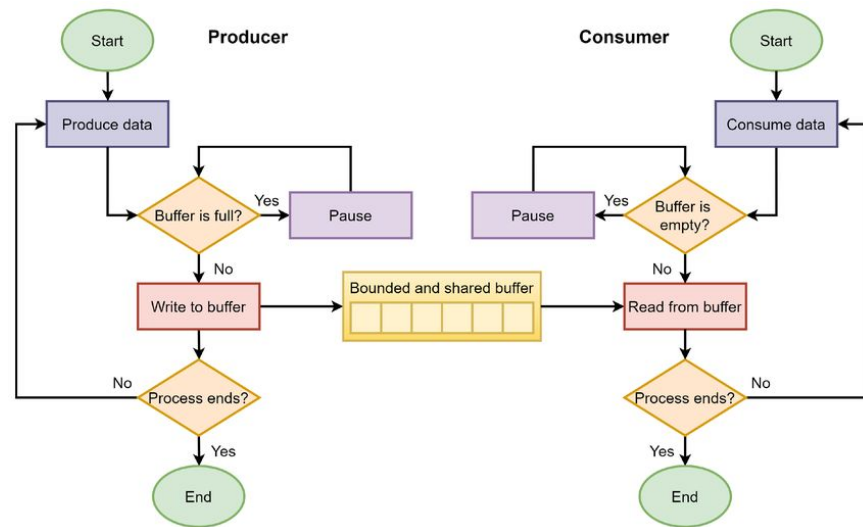
 **Semaphores Used** ($n = \text{buffer size}$):  **mutex** = 1 $\rightarrow$ Mutual exclusion,  **empty** = $n \rightarrow$ Empty slots in buffer and
 **full** = 0 $\rightarrow$ Filled slots in buffer

● Producer Code

```
while (true) {
    item = produceItem();
    wait(empty);    // Wait if buffer is full
    wait(mutex);    // Enter critical section
    buffer[in] = item;
    in = (in + 1) % n;
    signal(mutex);  // Exit critical section
    signal(full);   // One more full slot
}
```

● Consumer Code

```
while (true) {
    wait(full);     // Wait if buffer is empty
    wait(mutex);    // Enter critical section
    item = buffer[out];
    out = (out + 1) % n;
    signal(mutex);  // Exit critical section
    signal(empty);  // One more empty slot
    consumeItem(item);
}
```



The Dining Philosophers Problem

Problem Overview

- A **classic synchronization problem**
- Used to explain **deadlock, starvation, and resource sharing**
- Originally proposed as a **theoretical concurrency problem**

Scenario

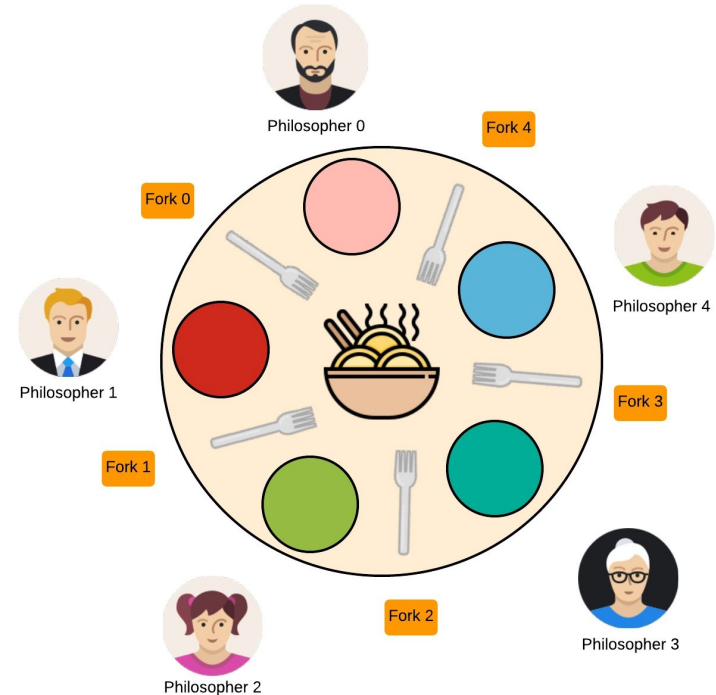
-  **5 philosophers** sit around a **circular table**
-  **5 forks**, one between each pair
-  Each philosopher alternates between: **Thinking and Eating**
-  To eat, a philosopher needs **both left & right chopsticks**
-  After eating, forks are released

Philosopher States

1.  **Thinking**: Not using forks
2.  **Hungry**: Tries to acquire forks
3.  **Eating**: Holds both forks

Key Issues Highlighted

-  **Mutual exclusion** of forks
-  Possible **deadlock**
-  Possible **starvation**
-  Fair resource allocation



Dining Philosophers – Key Challenges

Dining Philosophers' Dilemmas

1. Mutual Exclusion

-  Only **one philosopher** can hold a chopstick
- Chopsticks are **shared resources**
- Simultaneous use is **not allowed**

2. Deadlock

- All philosophers pick up **left chopstick**
- Each waits for **right chopstick**
-  No one can proceed → **System stuck**

3. Starvation

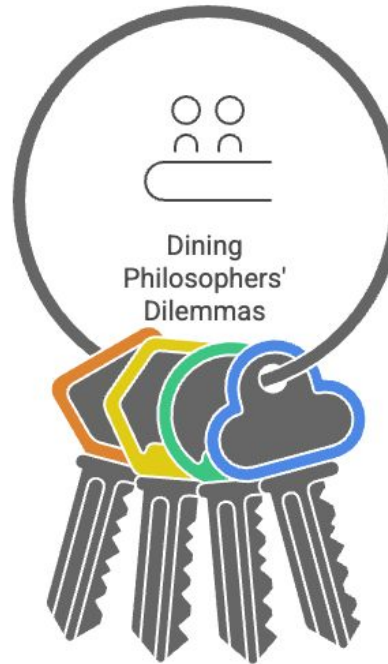
- A philosopher may **never get to eat**
- Neighbors keep eating alternately
- Leads to **unfair resource allocation**

4. Concurrent Access

- Multiple philosophers attempt to eat at once
- Requires **proper synchronization**
- Prevents conflicts and ensures progress

Deadlock Scenario (Example)

T1: All philosophers pick up **LEFT** chopstick
 T2: All try to pick up **RIGHT** chopstick
 → **DEADLOCK** (infinite **waiting**)



Mutual Exclusion

Ensures only one philosopher can hold a chopstick at a time.



Deadlock

Occurs when all philosophers wait for the right chopstick, causing a system freeze.



Starvation

A philosopher may never get to eat due to unfair resource allocation.



Concurrent Access

Requires proper synchronization to prevent conflicts when multiple philosophers eat.

Naive Semaphore Solution (Incorrect)

Approach

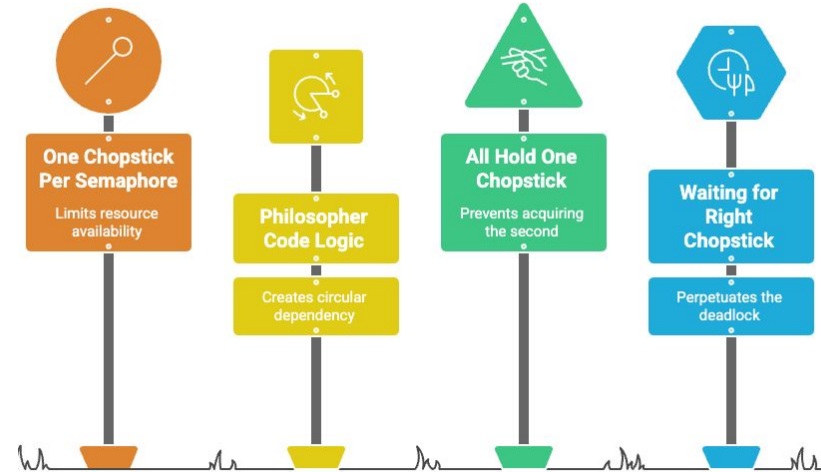
- One semaphore per chopstick
- Each chopstick initialized to 1
- semaphore `chopstick[5] = {1, 1, 1, 1, 1};`

Philosopher Code

```
void philosopher(int i) {  
    while (true) {  
        think();  
        wait(chopstick[i]);           // Pick left  
        wait(chopstick[(i+1) % 5]);  // Pick right  
        eat();  
        signal(chopstick[i]);        // Put left  
        signal(chopstick[(i+1) % 5]); // Put right  
    }  
}
```

Why This Fails

- **✗ Deadlock is possible**
- All philosophers may hold **one chopstick each**
- Each waits forever for the second one
- **!** Everyone waits for the **right chopstick** → **DEADLOCK**



Solution 1 – Asymmetric Solution

🧠 Strategy

- Use **one semaphore per chopstick**
- Make **one philosopher** pick chopsticks in **reverse order**
- Breaks **symmetry** in resource acquisition

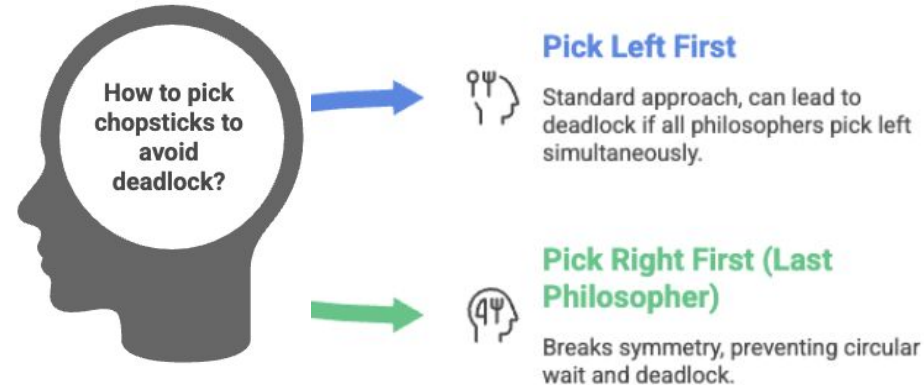
semaphore `chopstick[5] = {1, 1, 1, 1, 1};`

👤 Philosopher Code (Asymmetric)

```
void philosopher(int i) {
    while (true) {
        think();
        if (i == 4) {
            // Last philosopher picks RIGHT first
            wait(chopstick[(i+1) % 5]);
            wait(chopstick[i]);
        } else {
            // Others pick LEFT first
            wait(chopstick[i]);
            wait(chopstick[(i+1) % 5]);
        }
        eat();
        signal(chopstick[i]);
        signal(chopstick[(i+1) % 5]);
    }
}
```

✅ Why This Works

- Philosopher 4 picks chopsticks in **opposite order**
- **Circular wait condition is broken**
- Not all philosophers can hold one chopstick simultaneously
- **✓ Deadlock is prevented**



Solution 2 – Allow Maximum N-1 Philosophers

 **Strategy:** Allow at most 4 philosophers to try eating at the same time. Use an extra semaphore (**room**) to limit entry. Ensures at least one philosopher can eat.

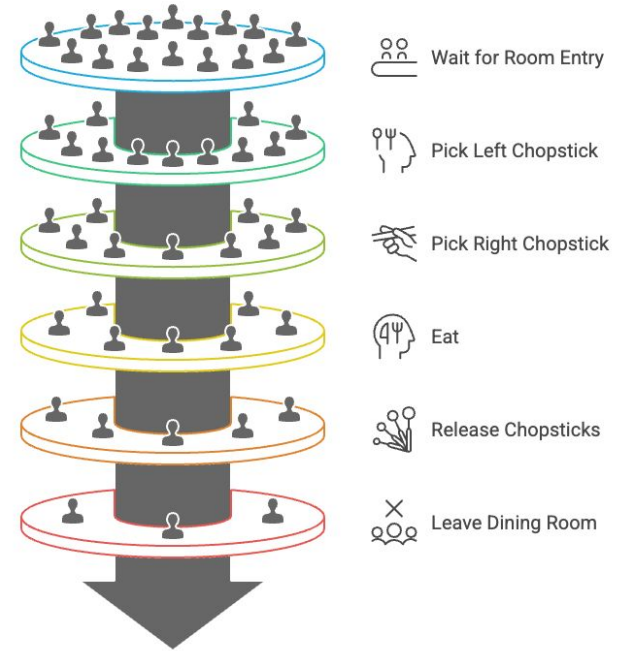
```
semaphore chopstick[5] = {1, 1, 1, 1, 1};
semaphore room = 4;    // Only 4 philosophers allowed
```

Philosopher Code

```
void philosopher(int i) {
    while (true) {
        think();
        wait(room);           // Enter dining room
        wait(chopstick[i]);   // Pick left chopstick
        wait(chopstick[(i+1) % 5]); // Pick right chopstick
        eat();
        signal(chopstick[i]);
        signal(chopstick[(i+1) % 5]);
        signal(room);         // Leave dining room
    }
}
```

Why This Works

- At most **N-1** philosophers compete for chopsticks
- At least **one** philosopher gets both chopsticks
- Circular wait is **impossible**
- Ensures **progress** (no deadlock)



Solution 3 – Using Mutex (All or Nothing)

🧠 **Strategy:** Use a **global mutex** to protect chopstick pickup. A philosopher must acquire **both chopsticks atomically**. Prevents partial allocation of resources.

```
semaphore chopstick[5] = {1, 1, 1, 1, 1};  
semaphore mutex = 1;    // Protects critical section
```

👤 Philosopher Code

```
void philosopher(int i) {  
    while (true) {  
        think();  
        wait(mutex);           // Enter critical section  
        wait(chopstick[i]);    // Pick left chopstick  
        wait(chopstick[(i+1) % 5]); // Pick right chopstick  
        signal(mutex);        // Exit critical section  
        eat();  
        wait(mutex);           // Enter critical section  
        signal(chopstick[i]);  // Put left chopstick  
        signal(chopstick[(i+1) % 5]); // Put right chopstick  
        signal(mutex);        // Exit critical section  
    }  
}
```

✓ **Pros:** No deadlock. Easy to understand and implement. Ensures **atomic resource acquisition**.

✗ **Cons:** Poor concurrency. Only **one philosopher** can pick chopsticks at a time. Reduces parallelism significantly.

How to pick up chopsticks?

Pick Left Chopstick

Acquires the left chopstick, ensuring atomicity with the right chopstick.

Pick Right Chopstick

Acquires the right chopstick, completing the pair for eating.



Solution 4 – Check Both Chopsticks (Tanenbaum's Solution)

 **Strategy:** A philosopher eats **only if both neighbors are not eating**. Chopsticks are picked up **together (logically)**. Uses **states + per-philosopher semaphore**.

Philosopher States

```
#define THINKING 0
#define HUNGRY 1
#define EATING 2
```

Shared Data Structures

```
int state[5]; // State of each philosopher
semaphore mutex = 1; // Protects critical section
semaphore s[5] = {0,0,0,0,0}; // One semaphore per philosopher
```

Helper Macros

```
#define LEFT (i+4) % 5
#define RIGHT (i+1) % 5
```

Solution 4 - Check Both Chopsticks (Tanenbaum's Solution)

🔍 Test Function (Core Logic)

```
void test(int i) {  
    if (state[i] == HUNGRY &&  
        state[LEFT] != EATING &&  
        state[RIGHT] != EATING) {  
        state[i] = EATING;  
        signal(s[i]); // Allow philosopher i  
to eat  
    }  
}
```

✅ Why This Works

- ✓ Philosopher eats **only when both chopsticks are free**
- ✓ **No deadlock** (no circular wait)
- ✓ **No starvation** (fair chance to eat)
- ✓ Allows **maximum safe concurrency**

Top Features of Tanenbaum's Solution



Tanenbaum's Solution – Main Functions

Take Chopsticks

```
void take_chopsticks(int i) {
    wait(mutex);
    state[i] = HUNGRY;
    test(i);      // Try to acquire both chopsticks
    signal(mutex);
    wait(s[i]);   // Block if not allowed to eat
}
```

Put Down Chopsticks

```
void put_chopsticks(int i) {
    wait(mutex);
    state[i] = THINKING;
    test(LEFT);   // Check left neighbor
    test(RIGHT);  // Check right neighbor
    signal(mutex);
}
```

 Sets Philosopher to HUNGRY

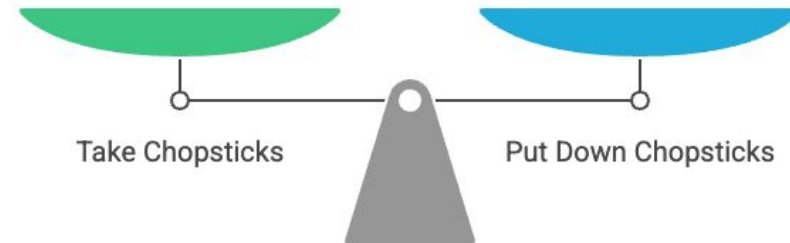
 Eats Only if Neighbors Not Eating

 Waits Safely

 Sets Philosopher to THINKING

 Wakes Up Neighbors if They Can Eat

 Releases Chopsticks



Tanenbaum's Solution – Main Functions

Philosopher Routine

```
void philosopher(int i) {  
    while (true) {  
        think();  
        take_chopsticks(i);  
        eat();  
        put_chopsticks(i);  
    }  
}
```

- Cycles through **Thinking** → **Hungry** → **Eating**
- Fully synchronized using semaphores

★ Advantages

- ✓ **No deadlock**
- ✓ **No starvation** (with fair semaphores)
- ✓ **Maximum parallelism**
- ✓ **Clean & structured design**

Problems with Semaphores

1. Scattered `wait()` / `signal()`

- Spread across the program
-  Hard to **verify correctness**
-  Easy to introduce bugs

2. Common Programming Errors

```
signal(mutex); ... wait(mutex);    // Mutual exclusion violated
wait(mutex); ... wait(mutex);      // Deadlock
wait(mutex); ... /* forget signal */ // Deadlock
```

- Small mistakes → **serious failures**

3. No Data Abstraction

- Synchronization mixed with logic
-  Poor readability
-  Difficult maintenance

4. Order Matters

- Incorrect order of `wait()` calls → **deadlock**
-  Bugs are **hard to debug**

5. Global Visibility

- Semaphores accessible everywhere
-  No encapsulation
-  Weak modular design

Problems with Semaphores

! Key Limitation

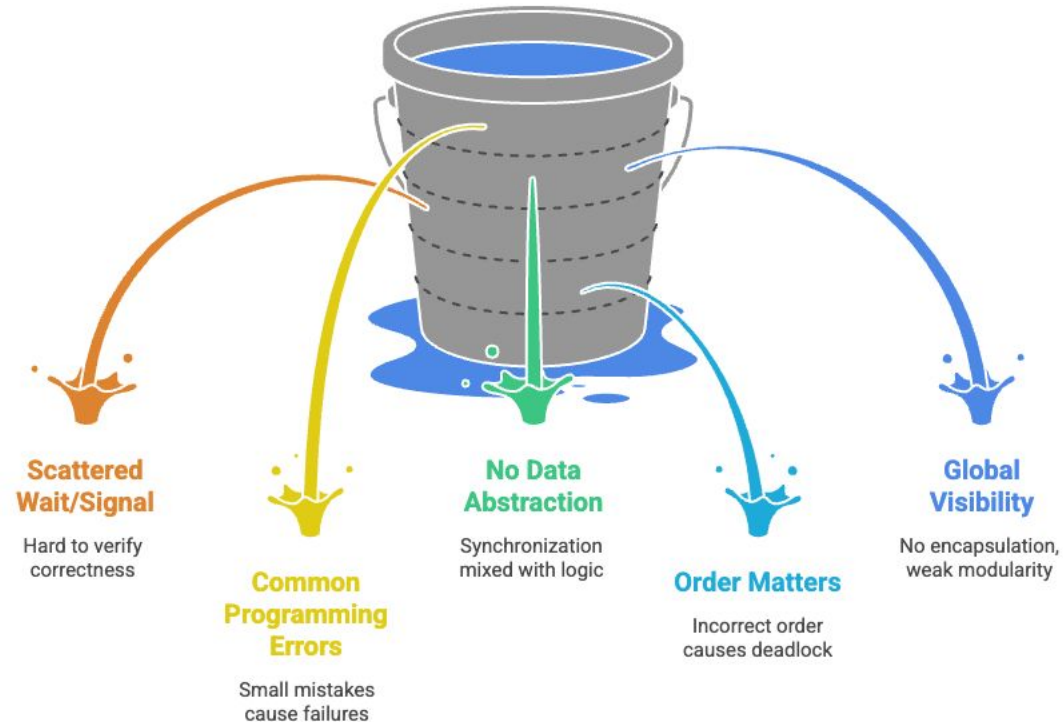
- Semaphores are **low-level & error-prone**

✓ Need for Better Solution

A synchronization construct that is:

- ✓ Easier to use
- ✓ Less error-prone
- ✓ Well-structured & modular

→ Leads to: Monitors & Condition Variables



Introduction to Monitors

It is a **high-level synchronization construct** that provides:

-  **Data Encapsulation** (like a class/module)
-  **Automatic Mutual Exclusion**
-  **Condition Synchronization** (via condition variables)

★ Key Characteristics

① Encapsulation

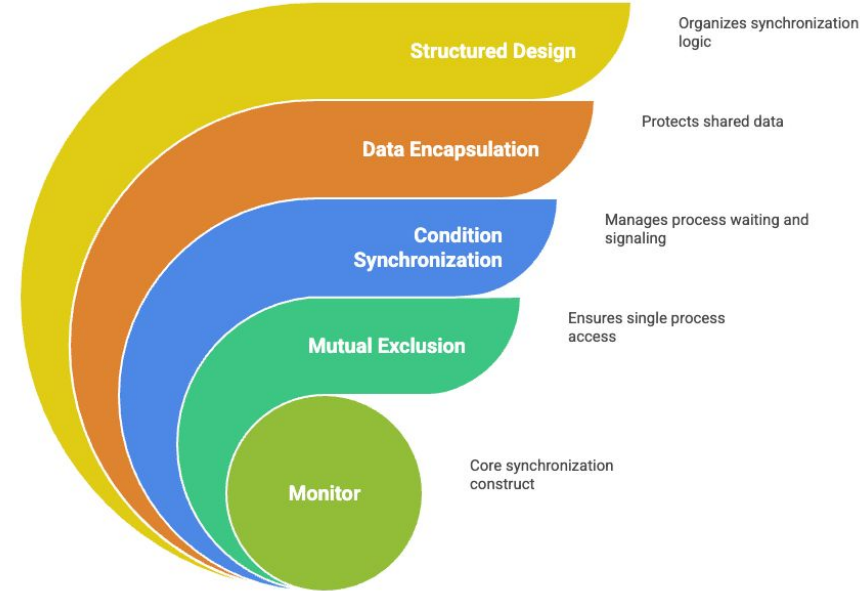
- Shared data is **private** to the monitor
- Access only through **monitor procedures**

② Mutual Exclusion

-  Only **ONE process** active inside the monitor at a time
- Enforced automatically by **compiler/runtime**

③ Structured Design

-  All synchronization logic in **one place**
- ✓ Clear separation of **logic vs synchronization**
- ✓ Easier to read, debug, and maintain



Monitor Syntax and Structure

Monitor Syntax (Pseudo-code)

```
monitor MonitorName {  
    // 1. SHARED DATA (private to monitor)  
    int sharedVariable1;  
    int sharedVariable2;  
    // 2. CONDITION VARIABLES (for synchronization)  
    condition conditionVar1;  
    condition conditionVar2;  
    // 3. INITIALIZATION (optional)  
    procedure init() {  
        sharedVariable1 = 0;  
        sharedVariable2 = 0;  
    }  
    // 4. ENTRY PROCEDURES (the only way to access shared data)  
    procedure entryProcedure1(parameters) {  
        // Code that operates on shared data  
        // Can use condition variables here  
    }  
    procedure entryProcedure2(parameters) {  
        // More operations  
    }  
}
```

Monitor Syntax and Structure

★ Key Rules

🔒 Automatic Mutual Exclusion

- 🧑 Only **ONE process** executes inside the monitor
- ❌ No explicit locks (`wait/signal`) required

⌚ Entry Queue

- Processes waiting to enter monitor are placed in an **entry queue**
- Scheduling decided by **system policy**

📊 Local vs Shared Data

- 📦 **Shared data** → protected inside monitor
- 📄 **Local variables** → private to each procedure call

🔄 Procedure Calls

- External processes call **monitor procedures**
- Similar to function calls
- 🔒 Mutual exclusion enforced automatically

Syntax and Structure

Defines the basic layout of a monitor



Entry Queue

Manages processes waiting to enter the monitor



Automatic Mutual Exclusion

Ensures only one process executes at a time



Local vs Shared Data

Distinguishes between private and protected data



Procedure Calls

Allows external processes to interact with the monitor



Condition Variables

What are Condition Variables?

- Special variables used for **synchronization inside a monitor**
- Allow processes to **wait for a condition to become true**
- Used with **monitors**, not standalone like semaphores

Operations

`wait(condition)`

- Process **releases monitor lock**
- Goes to **sleep** on condition's queue
- Another process can enter the monitor

`signal(condition)`

- Wakes up **ONE** waiting process
- If no process is waiting → **signal is lost**

Syntax (Pseudo-code)

```
condition x, y;    // Declaration
x.wait();          // Wait on condition x
x.signal();        // Signal condition x
```



Producer-Consumer Using Monitor

Monitor Definition (Pseudo-code)

```
monitor ProducerConsumer {  
    int buffer[N];  
    int count = 0, in = 0, out = 0;  
    condition notFull, notEmpty;  
    procedure insert(int item) {  
        while (count == N)    // Buffer full  
            notFull.wait();  
        buffer[in] = item;  
        in = (in + 1) % N;  
        count++;  
        notEmpty.signal();    // Wake consumer  
    }  
    procedure remove(int *item) {  
        while (count == 0)    // Buffer empty  
            notEmpty.wait();  
        *item = buffer[out];  
        out = (out + 1) % N;  
        count--;  
        notFull.signal();    // Wake producer  
    }  
}
```



Producer-Consumer Using Monitor

● Producer Code

```
while (true) {  
    item = produceItem();  
    ProducerConsumer.insert(item);  
}
```

● Consumer Code

```
while (true) {  
    ProducerConsumer.remove(&item);  
    consumeItem(item);  
}
```

★ Key Points

- **Mutual exclusion** handled automatically
- `notFull` → producer waits if buffer is full
- `notEmpty` → consumer waits if buffer is empty
- `while` loop used (Mesa semantics)

Semaphores vs Monitors Comparison

 Comparison Table

Aspect	Semaphores	Monitors
Abstraction Level	Low-level primitive	High-level construct
Mutual Exclusion	Manual (wait/signal)	Automatic
Data Protection	✗ Not enforced	✓ Encapsulation enforced
Error Prone	✓ High	✗ Low
Flexibility	More flexible	More restrictive
Code Structure	Scattered	Centralized
Debugging	Difficult	Easier
Language Support	Library-based	Language / compiler support
Efficiency	Generally faster	Slight overhead
Typical Usage	OS kernel, drivers	Application-level code



Use Semaphores when:

- Simple synchronization is required
- Performance is **critical**
- Writing **low-level system code** (OS kernel, drivers)



Use Monitors when:

- Synchronization is **complex**
- Application-level programming
- **Team development** (better readability & maintenance)

Implementing Monitors Using Semaphores

 **Key Idea: Monitors are high-level constructs.** Internally, they are implemented using **semaphores**. Ensures mutual exclusion and condition synchronization

Required Semaphores (Monitor-Level)

```
semaphore mutex = 1;    // Ensures mutual exclusion
semaphore next = 0;     // Controls signaling order
int next_count = 0;    // Number of suspended signalers
```

For Each Condition Variable *x*

```
semaphore x_sem = 0;    // Queue for condition x
int x_count = 0;       // Waiting process count
```

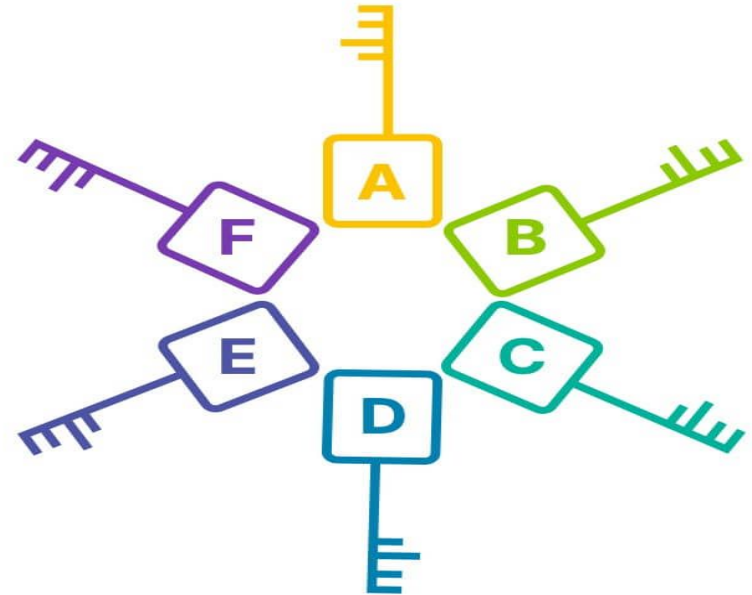
Monitor Entry / Exit

```
wait(mutex);
    // Monitor procedure body
signal(mutex);
```

- Ensures **only one process** is inside the monitor

Key Takeaways

- Review of Producer-Consumer
- Dining Philosophers Problem
- Semaphore Solutions for Philosophers
- Introduction to Monitors
- Condition Variables in Monitors
- Monitor Examples and Comparison



Thank You!

Thank you for your attention. Feel free to ask questions or revisit the topics covered.